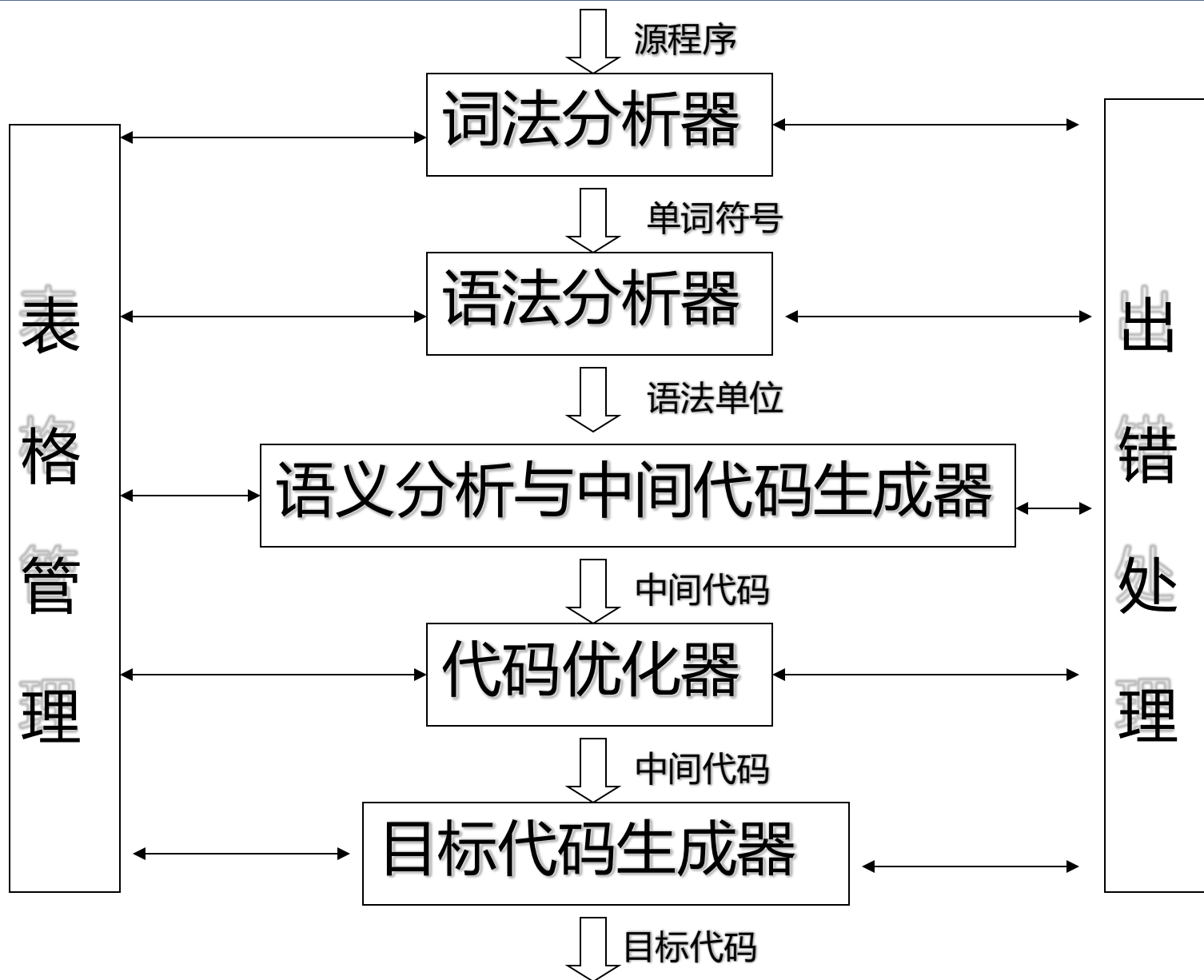




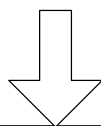
回顾 编译程序总体结构



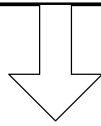


回顾 词法分析器的输入和输出

sum = (10+20)*(num+square)



词法分析器



(标识符, **sum**)(赋值号, =)(左括号, ()(整常数, **10**)(加号, +)(整常数, **20**)(右括号,))
(乘号, *) (左括号, ()(标识符, **num**)(加号, +)(标识符, **square**)(右括号,))(分号, ;)



第三章 词法分析

重点：词法分析器的输入、输出，用于识别符号的状态转移图的构造，根据状态转移图实现词法分析器。

难点：词法的正规文法表示、正规表达式表示、状态转移图表示，它们之间的转换。



第3章 词法分析

3.1 词法分析器的功能

3.2 单词的描述

3.3 单词的识别

3.4 词法分析程序的自动生成

3.5 本章小结



3.1 词法分析器的功能

□功能：输入源程序，输出单词符号(token)。即：把构成源程序的字符串转换成“等价的”单词(记号)序列

- 根据词法规则识别及组合单词，进行词法检查
- 对数字常数完成数字字符串到二进制数值的转换
- 删去空格字符和注释



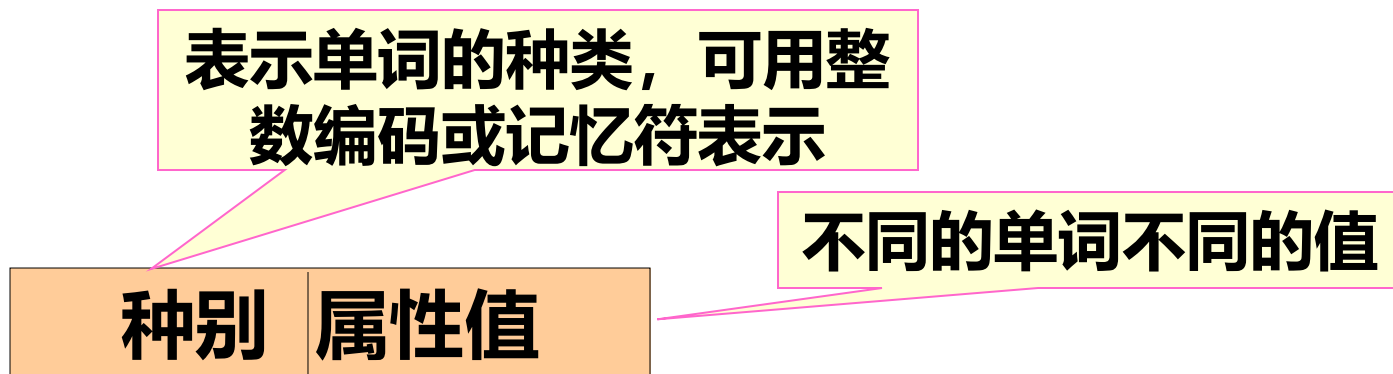
3.1.1 单词的分类与表示

一、单词的种类

1. 关键字——也称基本字, begin、end、for、do...
2. 标识符——由用户定义, 表示各种名字
3. 常数——整常数、实常数、布尔常数、字符串常数等
4. 运算符——算术运算符+、-、*、/等; 逻辑运算符not、or与and等; 关系运算符=、<>、>=、<=、>和<等
5. 分界符——, 、; 、(、) ...



3.1.2 单词的内部形式



几种常用的单词内部形式：

- 1、按单词种类分类
- 2、保留字和分界符采用一符一类
- 3、标识符和常数的单词值又为指示字
(指针值)



(1) 按单词种类分类

单词名称	类别编码	单词值
标识符	1	内部字符串
无符号常数(整)	2	整数值
无符号浮点数	3	数值
布尔常数	4	0 或 1
字符串常数	5	内部字符串
保留字	6	保留字或内部编码
分界符	7	分界符或内部编码



(2) 保留字和分界符采用一符一类

单词名称	类别编码	单词值
标识符	1	内部字符串
无符号常数(整)	2	整数值
无符号浮点数	3	数值
布尔常数	4	0 或 1
字符串常数	5	内部字符串
BEGIN	6	-
END	7	-
FOR	8	-
DO	9	-
.....		
:	20	-
+	21	-
*	22	-
,	23	-
(		-



例 语句if count>7 then result := 3.14; 的单词符号序列

(IF, 0)

(ID, 指向count的符号表入口)

(GT, 0)

(INT, 7)

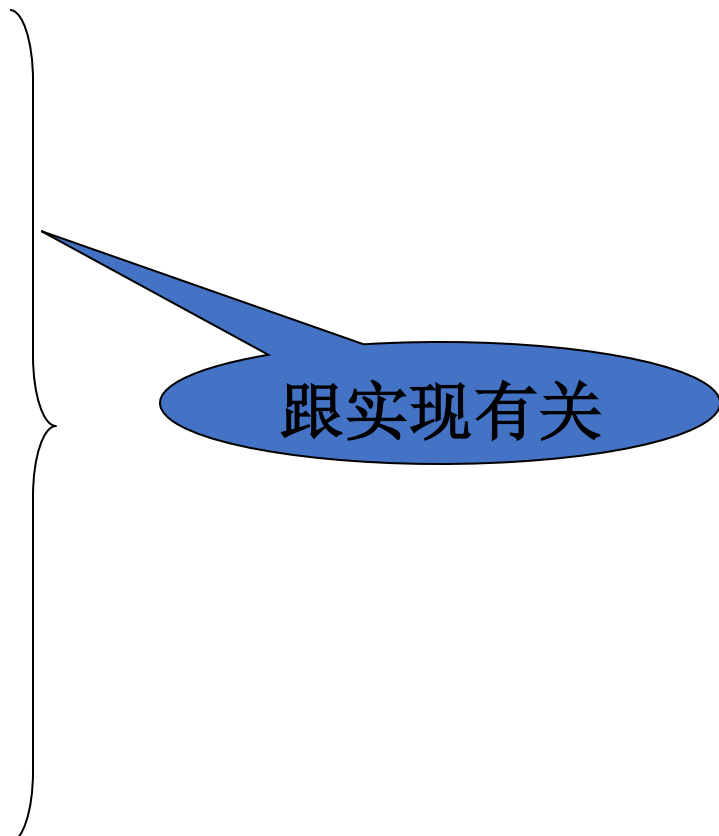
(THEN, 0)

(ID, 指向result的符号表入口)

(ASSIGN, 0)

(REAL, 3.14)

(SEMIC, 0)



跟实现有关



3.1.3 源程序的输入缓冲与预处理

□超前搜索和回退

- 双字符运算符 (`**`, `/*`, `:=`, ...)
- `DO 90 k=1,10`
- `DO 90 k=1.10`

□缓冲区

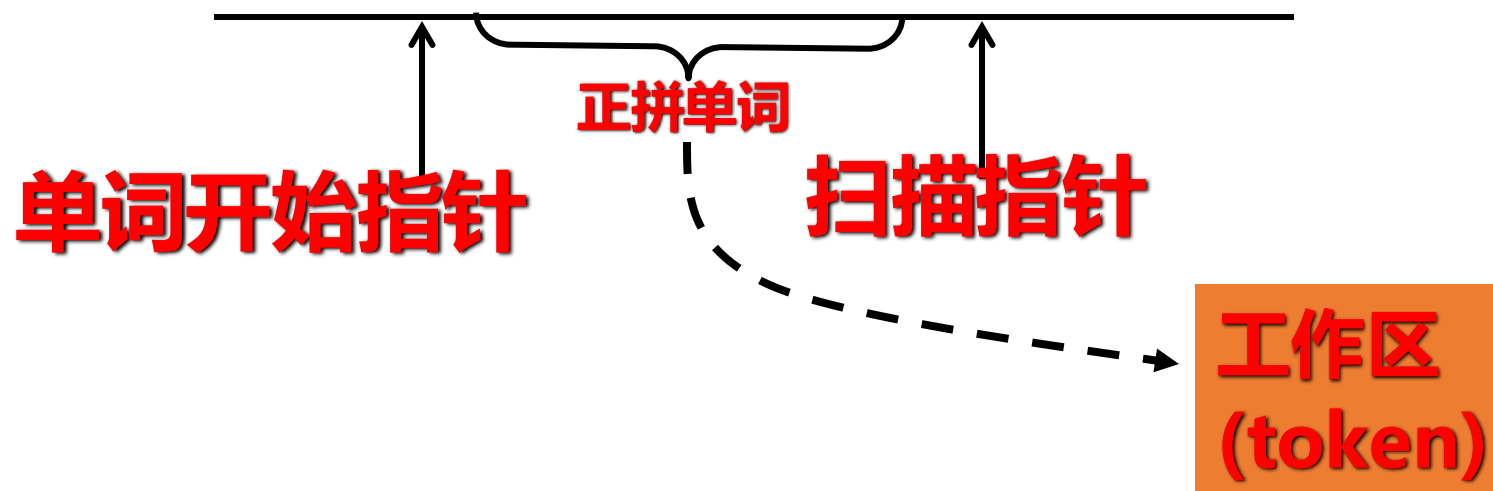
- 假定源程序存储在磁盘上，这样每读一个字符就需要访问一次磁盘，效率显然是很低的。

□空白字符的剔除

- 剔除源程序中的无用符号、空格、换行、注释等

3.1.3 源程序的输入缓冲与预处理

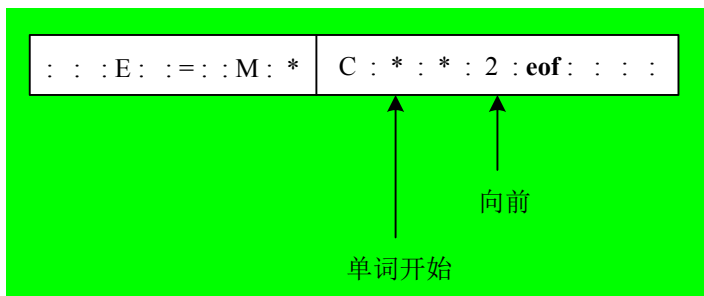
□输入缓冲区



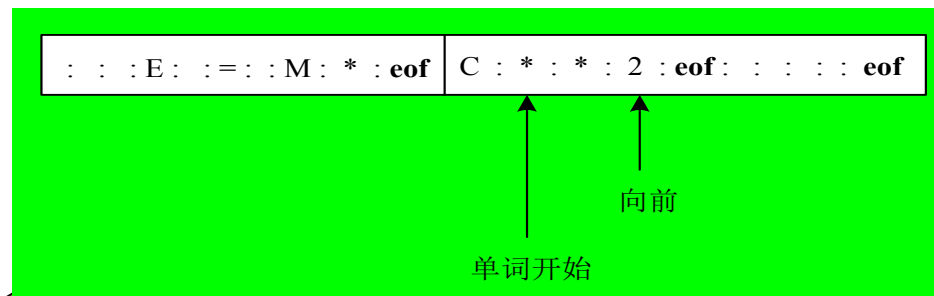


3.1.3 源程序的输入缓冲与预处理

□双缓冲区问题__超前扫描导致的效率问题



```
if forward在缓冲区第一部分末尾 then begin
  重装缓冲区第二部分;
  forward := forward + 1
end
else if forward在缓冲区第二部分末尾 then begin
  重装缓冲区第一部分;
  将forward移到缓冲区第一部分开始
end
else forward := forward + 1;
```



```
forward := forward + 1;
if forward↑= eof then begin
  if forward在第一部分末尾 then begin
    重装第二部分;
    forward := forward + 1
  end
  else if forward在第二部分末尾 then begin
    重装第一部分;
    将forward 移到第一部分开始
  end
end
else /* eof 在表示输入结束 */
  终止词法分析
end
```

- **大小问题**
128Byte*2|1024Byte*2|4096Byte*2

- **问题：如何设计和实现扫描器？**



3.1.4 词法分析阶段的错误处理

1. 非法字符检查

词法分析阶段的错误通常包括以下几种。

(1) 非法字符,即不包含在程序设计语言字符集中的字符,例如“@”对 Pascal 来说就是非法字符。

2. 关键字拼写错误检查

(2) 单词拼写错误,例如 `begen`(关键字 `begin` 拼写错误)、`3b78` 和 `4.5.5` 等。

3. 不封闭错误检查

(3) 注解或字符常数不封闭,例如 `/ * ... 'fixed...`。

4. 重复说明检查

(4) 变量重复说明,如 `integer x; real x。`

5. 错误恢复与续编译

紧急方式恢复(panic-mode recovery)

反复删掉剩余输入最前面的字符,直到词法分析器能发现一个正确的单词为止。



3.1.5 词法分析器的位置

□以词法分析器设计为独立的一遍的优点：

- 简化编译器的设计。
- 提高编译器的效率。
- 增强编译器的可移植性。

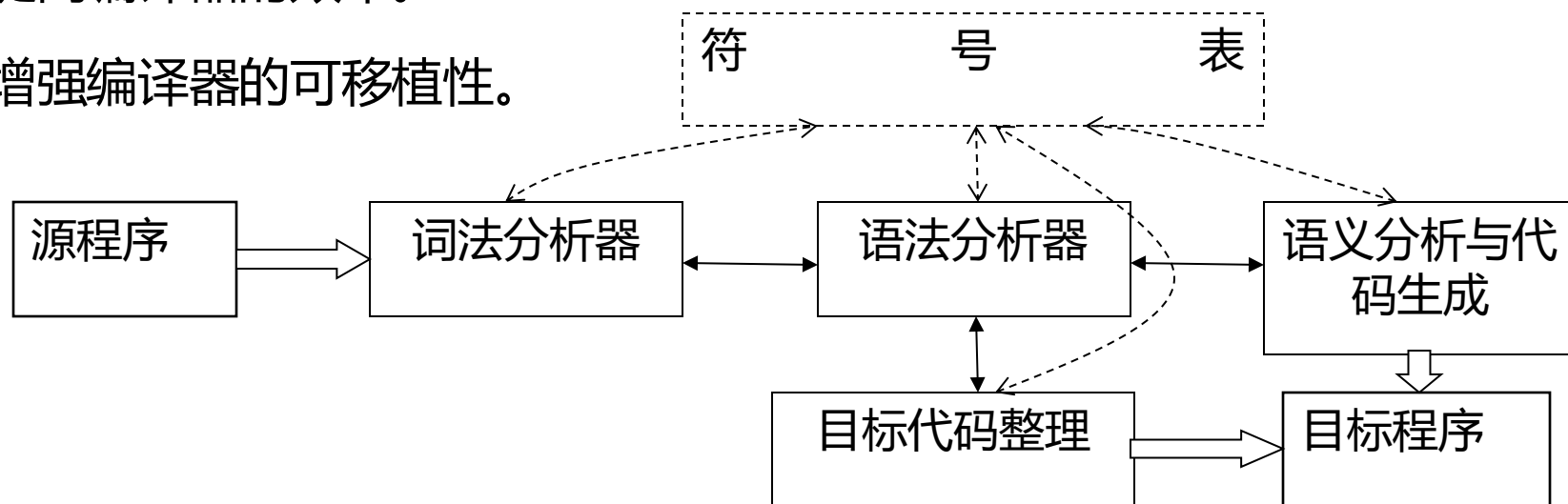


图 以语法分析器为中心

词法分析器的输出结果是()。

- ☒ A 单词的种别编码和自身值
- ☐ B 单词在符号表中的位置
- ☐ C 单词的种别编码
- ☐ D 单词自身值

提交



3.2 单词的描述

3.2.1 正则文法

□正则文法 $G = (V, T, P, S)$ 中, 对 $\forall \alpha \rightarrow \beta \in P$, $\alpha \rightarrow \beta$ 均具有形式 $A \rightarrow w$ 或 $A \rightarrow wB$ ($A \rightarrow w$ 或 $A \rightarrow Bw$), 其中 $A, B \in V$, $w \in T^+$ 。

□例3.1 标识符的文法

- 约定: 用digit表示数字: 0,1,2,...,9;
用letter表示字母: A,B,...,Z,a,b,...,z
- $\langle \text{id} \rangle \rightarrow \langle \text{letter} \rangle \mid \langle \text{id} \rangle \langle \text{digit} \rangle \mid \langle \text{id} \rangle \langle \text{letter} \rangle$
- $\langle \text{letter} \rangle \rightarrow A \mid B \mid \dots \mid Z \mid a \mid b \mid \dots \mid z$
- $\langle \text{digit} \rangle \rightarrow 0 \mid 1 \mid 2 \mid \dots \mid 9$



3.2.2 正则表达式

□例3.2：标识符的另一种表示

- $\text{letter}(\text{letter}|\text{digit})^*$
- | 表示"或"
- * 表示Kleene闭包
- + 表示正闭包
- ? 表示 "0或1个"
- letter 和 $(\text{letter}|\text{digit})^*$ 的并列表示两者的连接

□正则式 r 表示正则集,相应的正则集记为 $L(r)$



3.2.2 正则表达式

定义3.1 设 Σ 是一个字母表，则 Σ 上的正则表达式及其所表示的正则语言可递归地定义如下：

- (1) $\emptyset$ 是 Σ 上的一个正则表达式，它表示空集；
- (2) ε 是 Σ 上的一个正则表达式，它表示语言 $\{\varepsilon\}$ ；
- (3) 对于 $\forall a(a \in \Sigma)$ ， a 是 Σ 上的一个正则表达式，它表示的正则语言是 $\{a\}$ ；
- (4) 假设 r 和 s 都是 Σ 上的正则表达式，它们表示的语言分别为 $L(r)$ 和 $L(s)$ ，则：
 - ① (r) 也是 Σ 上的正则表达式，它表示的语言为 $L(r)$ ；
 - ② $(r|s)$ 也是 Σ 上的正则表达式，它表示的语言为 $L(r) \cup L(s)$ ；
 - ③ $(r \cdot s)$ 也是 Σ 上的正则表达式，它表示的语言为 $L(r)L(s)$ ；
 - ④ (r^*) 也是 Σ 上的正则表达式，它表示的语言为 $(L(r))^*$ ；
- (5) 只有经有限次使用上述规则构造的表达式才是 Σ 上的正则表达式。



正则表达式中的运算优先级

运算优先级和结合性:

- *高于“连接”和|, “连接”高于|
- |具有交换律、结合律
- “连接”具有结合律、和对|的分配律
- ()指定优先关系

意义清楚时, 括号可以省略

例:

1. $L((a|b)^*) = L((a^*|b^*)^*) = \{x | x \text{ 是 } a \text{ 和 } b \text{ 构成的符号串, 包括 } \varepsilon\}$

2. $L(a|a^*b) = \{a, b, ab, aab, aaab, aaaab, \dots\}$



3.2.3 正则表达式与正则文法的等价性

□正则表达式与正则文法等价

- 对任意一个正则表达式，存在一个定义同一语言的正则文法
- 对任意一个正则文法，存在一个定义同一语言的正则表达式

证明语言的性质：正则文法

实践、设计自动机：正则表达式



根据正则文法构造等价的正则表达式

□问题：给定正则文法 G ，构造一个正则表达式 r ，使得 $L(r) = L(G)$

□基本思路

- 为正则文法的每个产生式构造一个正则表达式方程式，这些方程式中的变量是文法 G 中的语法变量，各变量的系数是正则表达式，简称为方程式。从而得到一个联立方程组。
- 用代入消元法消去联立方程组中除开始符号外的其他变量，最后得到关于开始符号 S 的解： $S = r$ ， r 即为所求的正则表达式。



根据正则文法构造等价的正则表达式

□具体步骤

(1) 根据正则文法 G 构造正则表达式联立方程组。

假设正则文法 G 是右线性的，其每个产生式的右部只含有一个终结符，则有如下方程式构造规则：

- ① 对形如 $A \rightarrow a_1 | a_2 | \dots | a_m$ 的产生式，构造方程式 $A = a_1 | a_2 | \dots | a_m$ 。其中可以有形如 $A \rightarrow \varepsilon$ 的产生式；
- ② 对形如 $A \rightarrow a_1 A | a_2 A | \dots | a_m A$ 的产生式，构造方程式 $A = (a_1 | a_2 | \dots | a_m)^* A$ ；
- ③ 对形如 $A \rightarrow a_1 B | a_2 B | \dots | a_m B$ 的产生式，构造方程式 $A = (a_1 | a_2 | \dots | a_m) B$ ，其中 $B \neq A$ 。

根据正则文法构造等价的正则表达式

(2)解联立方程组，求等价的正则表达式 r

用代入消元法逐个消去方程组中除开始符号 S 外的其他变量，最后即可得到关于开始符号 S 的解。代入消元规则如下：

① 如果有 $A=r_1B|r_2B|\dots|r_nB$ ，则用 $A=(r_1|r_2|\dots|r_n)B$ 替换之，其中 $B\neq A$ ；

② 如果有 $A=t_1A|t_2A|\dots|t_mA$ ，则用 $A=(t_1|t_2|\dots|t_m)^*A$ 替换之；



根据正则文法构造等价的正则表达式

③ 如果有 $A=(r_1|r_2|\dots|r_n)B$, $B=(t_1|t_2|\dots|t_m)C$, 则用 $A=(r_1|r_2|\dots|r_n) (t_1|t_2|\dots|t_m)C$ 替换之, 其中 $B \neq A$;

如果有 $A=(r_1|r_2|\dots|r_n)B$, $B=(t_1|t_2|\dots|t_m)$, 则用 $A=(r_1|r_2|\dots|r_n) (t_1|t_2|\dots|t_m)$ 替换之, 其中 $B \neq A$;

④ 对 $A=(t_1|t_2|\dots|t_m)^*A$ 且 $A=(r_1|r_2|\dots|r_n)B$, 其中 $B \neq A$, 则用 $A=(t_1|t_2|\dots|t_m)^* (r_1|r_2|\dots|r_n)B$ 替换之;

对 $A=(t_1|t_2|\dots|t_m)^*A$ 且 $A=r_1|r_2|\dots|r_n$ 则用 $A=(t_1|t_2|\dots|t_m)^* (r_1|r_2|\dots|r_n)$ 替换之;

⑤ 如果 $A=\beta_1$ 、 $A=\beta_2$ 、...、 $A=\beta_h$, 则用 $A=\beta_1|\beta_2|\dots|\beta_h$ 代替之。



根据正则文法构造等价的正则表达式

如果最后得到的关于 S 的方程式为如下形式,

$$S = \alpha_1 | \alpha_2 | \dots | \alpha_h$$

则将方程式右边所有其中仍然含有语法变量的 $\alpha_i (1 \leq i \leq h)$ 删除, 得到的结果就是与 G 等价的正则表达式; 如果任意的 $\alpha_i (1 \leq i \leq h)$ 均含有语法变量, 则 $\emptyset$ 就是与 G 等价的正则表达式。

根据正则文法构造等价的正则表达式

□例3.6 将如下文法 G 转换成相应的正则表达式

$$S \rightarrow aS|aB$$

$$B \rightarrow bB|bC|aB|bS$$

$$C \rightarrow cC|c$$

1.列方程组

- $S=a^*S \quad S=aB \quad B=(a|b)^*B \quad B=bC$
- $B=bS \quad C=c^*C \quad C=c$

2.代入法解方程组

- $C=c^*c; \quad B=bc^*c|bc; \quad B=(a|b)^*(bc^*c|bc); \quad B=(a|b)^*bS;$
- $S=a^*aB;$
- $S=a^*a(a|b)^*bS|a^*a(a|b)^*(bc^*c|bc);$
- $S=(a^*a(a|b)^*b)^*a^*a(a|b)^*(bc^*c|bc);$
- 如果用正闭包表示, 则为 $(a^+(a|b)^*b)^*a^+(a|b)^*(bc^+|bc)$



根据正则文法构造等价的正则表达式

例 3.6 将如下的文法 G 转换成相应的正则表达式。

$$S \rightarrow aS \mid aB$$

$$B \rightarrow bB \mid bC \mid aB \mid bS$$

$$C \rightarrow cC \mid c$$

解：

(1) 根据方程式构造规则分别构造与各个产生式对应的方程式。

根据产生式 $S \rightarrow aS$ 和方程式构造规则②得到如下方程式：

$$S = a^* S$$

根据产生式 $S \rightarrow aB$ 和规则③得到如下方程式：

$$S = aB$$

根据产生式 $B \rightarrow bB \mid aB$ 和方程式构造规则②得到如下方程式：

$$B = (a \mid b)^* B$$

根据产生式 $B \rightarrow bC$ 和方程式构造规则③得到如下方程式：

$$B = bC$$

根据产生式 $B \rightarrow bS$ 和方程式构造规则③得到如下方程式：

$$B = bS$$

根据产生式 $C \rightarrow cC$ 和方程式构造规则②得到如下方程式：

$$C = c^* C$$

根据产生式 $C \rightarrow c$ 和方程式构造规则①得到如下方程式：

假设正则文法 G 是右线性的,其每个产生式的右部只含有一个终结符,则有如下方程式构造规则 (equation construction rule)。

① 对形如 $A \rightarrow a_1 \mid a_2 \mid \cdots \mid a_m$ 的产生式,构造方程式 $A = a_1 \mid a_2 \mid \cdots \mid a_m$,其中可以有形如 $A \rightarrow \varepsilon$ 的产生式。

② 对形如 $A \rightarrow a_1 A \mid a_2 A \mid \cdots \mid a_m A$ 的产生式,构造方程式 $A = (a_1 \mid a_2 \mid \cdots \mid a_m)^* A$ 。

③ 对形如 $A \rightarrow a_1 B \mid a_2 B \mid \cdots \mid a_m B$ 的产生式,构造方程式 $A = (a_1 \mid a_2 \mid \cdots \mid a_m) B$,其中 $B \neq A$ 。



根据正则文法构造等价的正则表达式

$$C = c$$

(2) 用代入消元法消除 S 以外的语法变量。

根据方程式 $C = c^*C$ 、 $C = c$ 和代入消元规则④得到如下方程式：

$$C = c^*c$$

根据方程式 $C = c^*c$ 、 $C = c$ 、 $B = bC$ 和代入消元规则④、⑤得到如下方程式：

$$B = bc^*c | bc$$

根据方程式 $B = (a|b)^*B$ 、 $B = bc^*c | bc$ 和代入消元规则④以及 $(a|b)^* = (a|b)^*$ 得到如下方程式：

$$B = (a|b)^*(bc^*c | bc)$$

根据方程式 $B = (a|b)^*B$ 、 $B = bS$ 和代入消元规则④以及 $a^* = a^*$ 得到如下方程式：

$$B = (a|b)^*bS$$

根据方程式 $S = a^*S$ 、 $S = aB$ 和代入消元规则④得到如下方程式：

$$S = a^*aB$$

根据方程式 $S = a^*aB$ 、 $B = (a|b)^*bS$ 、 $B = (a|b)^*(bc^*c | bc)$ 和代入消元规则④、⑤得到如下方程式：

$$S = a^*a(a|b)^*bS | a^*a(a|b)^*(bc^*c | bc)$$

根据方程式 $S = a^*a(a|b)^*bS | a^*a(a|b)^*(bc^*c | bc)$ 和代入消元规则④得到如下方程式：

$$S = (a^*a(a|b)^*b)^*a^*a(a|b)^*(bc^*c | bc)$$

$(a^*a(a|b)^*b)^*a^*a(a|b)^*(bc^*c | bc)$ 就是与所给文法等价的正则表达式, 如果用正闭包表示, 则具有如下形式：

$$(a^*(a|b)^*b)^*a^*(a|b)^*(bc^* | bc)$$

① 如果有 $A = r_1B | r_2B | \dots | r_nB$, 则用 $A = (r_1 | r_2 | \dots | r_n)B$ 进行替换, 其中 $B \neq A$ 。

② 如果有 $A = t_1A | t_2A | \dots | t_mA$, 则用 $A = (t_1 | t_2 | \dots | t_m)^*A$ 进行替换。

③ 如果有 $A = (r_1 | r_2 | \dots | r_n)B$ 、 $B = (t_1 | t_2 | \dots | t_m)C$, 则用 $A = (r_1 | r_2 | \dots | r_n)(t_1 | t_2 | \dots | t_m)C$ 进行替换, 其中 $B \neq A$; 如果有 $A = (r_1 | r_2 | \dots | r_n)B$ 、 $B = (t_1 | t_2 | \dots | t_m)$, 则用 $A = (r_1 | r_2 | \dots | r_n)(t_1 | t_2 | \dots | t_m)$ 进行替换, 其中 $B \neq A$ 。

④ 对 $A = (t_1 | t_2 | \dots | t_m)^*A$ 且 $A = (r_1 | r_2 | \dots | r_n)B$, 其中 $B \neq A$, 则用 $A = (t_1 | t_2 | \dots | t_m)^*(r_1 | r_2 | \dots | r_n)B$ 进行替换; 对 $A = (t_1 | t_2 | \dots | t_m)^*A$ 且 $A = r_1 | r_2 | \dots | r_n$, 则用 $A = (t_1 | t_2 | \dots | t_m)^*(r_1 | r_2 | \dots | r_n)$ 进行替换。

⑤ 如果有 $A = \beta_1$ 、 $A = \beta_2$ 、 $\dots$ 、 $A = \beta_h$, 则用 $A = \beta_1 | \beta_2 | \dots | \beta_h$ 进行替换。

将正则表达式转换成等价的正则文法

□问题：给定 Σ 上的一个正则表达式 r ，根据 r 构造正则文法 G ，使得 $L(G)=L(r)$

□定义3.3 设字母表为 Σ ， $\{A、B、...、C\}$ 为语法变量集合，对于 Σ 上的任意正则表达式 r ，形如 $A \rightarrow r$ 的式子称为正则定义式；如果 r 是 Σ 中的字母和用正则定义式定义的变量组成的正则表达式，则形如 $A \rightarrow r$ 的式子称为正则定义式。

将正则表达式转换成等价的正则文法

□ 按如下方法构造正则定义式，并逐步将其转换成正则文法

□ 引入开始符号 S ，从如下正则定义式开始

$$S \rightarrow r$$

□ 按如下规则将 $S \rightarrow r$ 分解为新的正则定义式，在分解过程中根据需要引入新的语法变量

将正则表达式转换成等价的正则文法

□ $A \rightarrow r$ 是正则定义式, 则对 $A \rightarrow r$ 的分解规则如下:

- (1) 如果 $r = r_1 r_2$, 则将 $A \rightarrow r$ 分解为 $A \rightarrow r_1 B$, $B \rightarrow r_2$, $B \in V$;
- (2) 如果 $r = r_1^* r_2$, 则将 $A \rightarrow r$ 分解为 $A \rightarrow r_1 A$, $A \rightarrow r_2$;
- (3) 如果 $r = r_1 | r_2$, 则将 $A \rightarrow r$ 分解为 $A \rightarrow r_1$, $A \rightarrow r_2$ 。

不断应用分解规则(1)到(3)对各个正则定义式进行分解, 直到每个正则定义式右端只含一个语法变量(即符合正则文法产生式的形式)为止。

例3.9 正则表达式到正则文法的转换

例 3.8 将正则表达式 $a(a|b)^*$ 转换成相应的正则文法。 (1) 如果 $r=r_1r_2$, 则将 $A \rightarrow r$ 分解为 $A \rightarrow r_1B$, $B \rightarrow r_2$, $B \in V$;

解: 令 S 为文法的开始符号, 并构造正则定义式:

$$S \rightarrow a(a|b)^*$$

根据规则(1), 引入语法变量 A , 将 $S \rightarrow a(a|b)^*$ 分解为: (2) 如果 $r=r_1^*r_2$, 则将 $A \rightarrow r$ 分解为 $A \rightarrow r_1A$, $A \rightarrow r_2$;

$$S \rightarrow aA$$

$$A \rightarrow (a|b)^*$$

$S \rightarrow aA$ 的右端只含一个语法变量, 已经符合正则文法产生式的要求。对于正则定义式 $A \rightarrow$

$(a|b)^*$, 根据规则(2)可分解为:

$$A \rightarrow (a|b)A$$

$$A \rightarrow \varepsilon$$

最后按照连接对 $|$ 运算的分配律, 将 $A \rightarrow (a|b)A$ 转换为:

$$A \rightarrow aA | bA$$

至此就得到了与 $a(a|b)^*$ 相对应的正则文法:

$$S \rightarrow aA$$

$$A \rightarrow aA | bA | \varepsilon$$

□



例 3.10 标识符定义的转换

□引入 S

$$S \rightarrow \text{letter} (\text{letter}|\text{digit})^*$$

□分解为

$$S \rightarrow \text{letter } A$$

$$A \rightarrow (\text{letter}|\text{digit})A|\varepsilon$$

□执行连接对|的分配律

$$S \rightarrow \text{letter } A$$

$$A \rightarrow \text{letter } A|\text{digit } A|\varepsilon$$

与 $(a|b)^*(a|b)$ 等价的正规式是()。

- ☐ A $a^*|b^*$
- ☐ B $(ab)^*(a|b)$
- ☒ C $(a|b)(a|b)^*$
- ☐ D $(a|b)^*$

同正规式 a^*b^* 等价的文法是 () 。

- ☐ A $S \rightarrow aSb \mid \varepsilon$
- ☒ B $S \rightarrow aS \mid Sb \mid \varepsilon$
- ☐ C $S \rightarrow aS \mid bS \mid \varepsilon$
- ☐ D $S \rightarrow abS \mid \varepsilon$

提交



高级语言词法的简单描述

□词法

- 单词符号的文法，用来描述高级语言中的：
- 标识符、常数、运算符、分界符、关键字

□参考教材P73-77，了解如何定义高级语言中的整数、实数.....等的相应正则文法。

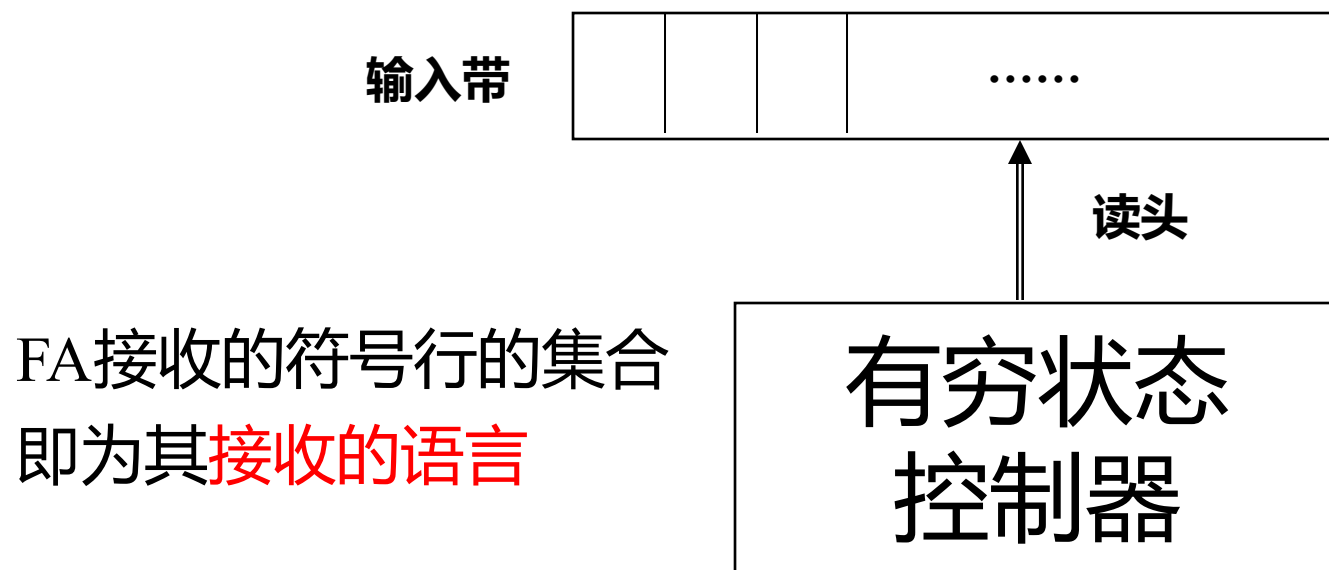


3.2.4 有穷状态自动机

- 具有离散输入输出的系统的数学模型
- 具有有穷个内部状态
- 系统只需根据当前所处的状态和面临的输入就能确定后继的行为，处理完当前输入后系统的状态将发生变化
- 具有初始状态和终止状态
- 例：电梯、文本编辑程序、词法分析程序.....



有穷自动机的物理模型



设想有个按钮，自动机启动后一个动作一个动作地做下去，...，直到没有输入。如果停在终态，接收；如果停在非终态，不接收

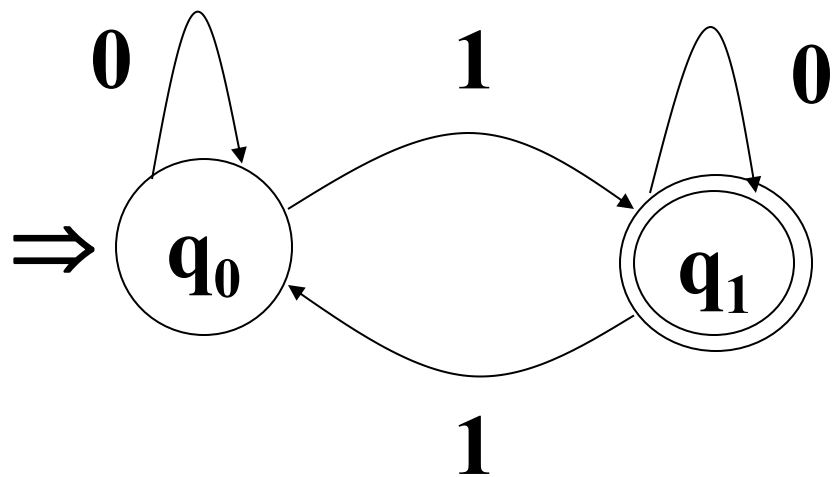
一个动作：

$[p, a] \rightarrow q$ ，读头前进一格



例：一个奇偶校验器

测试输入中1的个数的奇偶性，并且只接收含有奇数个1的那些输入串。



注意：状态有记忆功能，记住输入串的部分特征。

问题：有穷自动机的形式描述？

关键是如何描述动作？



确定的有穷自动机的形式定义

定义3.4 一个**确定的有穷自动机** M (记作DFA M) 是一个五元组

$M = (Q, \Sigma, \delta, q_0, F)$, 其中

- (1) Q 是一个有穷状态集合。
- (2) Σ 是一个字母表, 它的每个元素称输入符号。
- (3) $q_0 \in Q$, q_0 称为初始状态。
- (4) $F \subseteq Q$, F 称为终止状态集合。
- (5) δ 是一个从 $Q \times \Sigma$ 到 Q 的单值映射 $\delta(p, a) = q$ ($p, q \in Q, a \in \Sigma$)

表示当前状态为 p , 输入符号为 a 时, 自动机将转换到下一个状态 q , q 称为 p 的一个后继。



DFA的表示

例 设DFA $M = (\{0, 1, 2, 3\}, \{a, b\}, \delta, 0, \{3\})$

其中:

$$\delta(0, a) = 1, \delta(1, a) = 3$$

$$\delta(2, a) = 1, \delta(3, a) = 3$$

$$\delta(0, b) = 2, \delta(1, b) = 2$$

$$\delta(2, b) = 3, \delta(3, b) = 3$$

□一个DFA有三种表示:

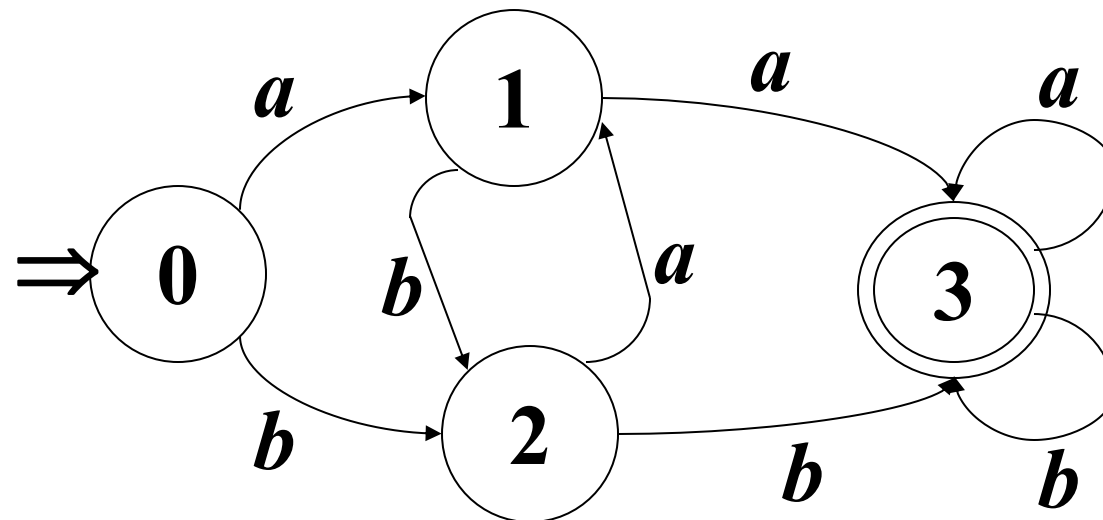
- (1) 转换函数;
- (2) 转移矩阵;
- (3) 状态转换图。

转移矩阵

	a	b
0	1	2
1	3	2
2	1	3
3	3	3

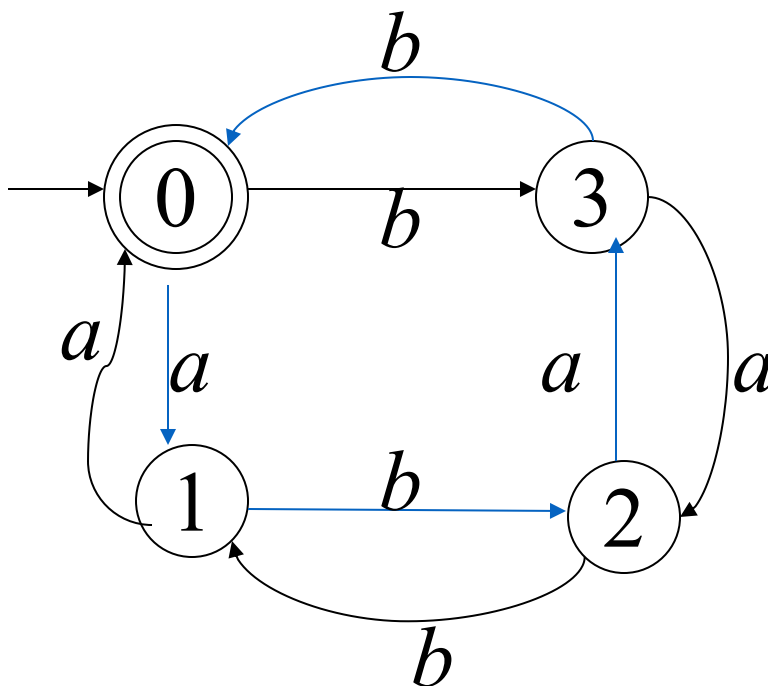
易存储

状态转换图





DFA M 接受的语言



从状态转换图看，从初态出发，沿任一条路径到达接受状态，这条路径上的弧上的标记符号连接起来构成的符号串被接受。

如： $abab$

问题：如何形式描述DFA接收的语言？



DFA M 接受的语言

如果对所有 $w \in \Sigma^*$, $a \in \Sigma$, $q \in Q$ 以下述方式递归地扩展 δ 的定义 $\hat{\delta}(q, \varepsilon) = q$,

$$\hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a),$$

则 M 所接收的语言为:

$$L(M) = \{w \mid w \in \Sigma^*, \text{ 且 } \delta(q_0, w) \in F\}$$

对于上页例中的 DFA M 和 $w=baa$,

$$\delta(0, baa) = \delta(2, aa) = \delta(1, a) = 3$$



DFA M 的模拟算法

输入：以eof结尾的串 x , DFA $M = (Q, \Sigma, \delta, q_0, F)$;

输出：如果 M 接受 x 则输出 “yes”, 如果 M 不接受 x 则输出 “no”

步骤：

1 $s = q_0$;

2 $c = \text{getchar}(x)$;

3 while ($c \neq \text{eof}$) {

4 $s = \text{move}(s, c)$;

5 $c = \text{getchar}(x)$;

6 }

7 if $s \in F$ return “yes”

8 else return “no”;



3.2.5 状态转换图

□定义3.7 设 $M=(Q, \Sigma, \delta, q_0, F)$ 为一个有穷状态自动机, 满足如下条件的有向图被称为 M 的状态转换图(transition diagram):

- (1) $q \in Q \Leftrightarrow q$ 是该有向图中的一个顶点;
- (2) $\delta(q, a)=p \Leftrightarrow$ 图中有一条从顶点 q 到顶点 p 的标记为 a 的弧;
- (3) $q \in F \Leftrightarrow$ 标记为 q 的顶点被用双层圈标出;
- (4) 用标有start的箭头指出 M 的开始状态。



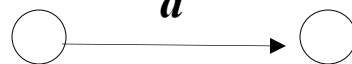
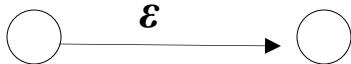
3.2.6 正则表达式转换为状态转换图



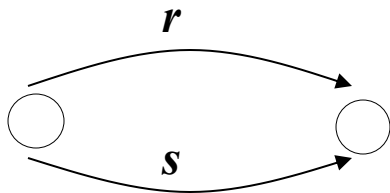
(a) $\emptyset$ 对应的状态转换图



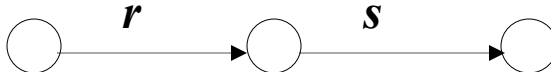
(b) ε 对应的状态转换图



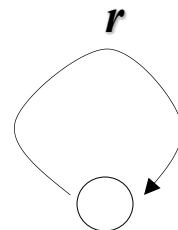
(c) a 对应的状态转换图



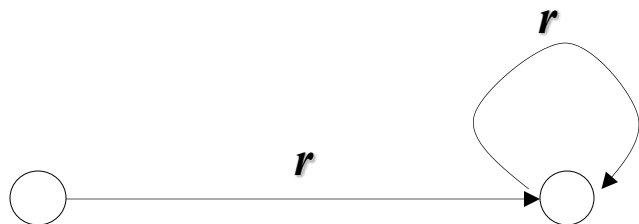
(d) $r \mid s$ 对应的状态转换图



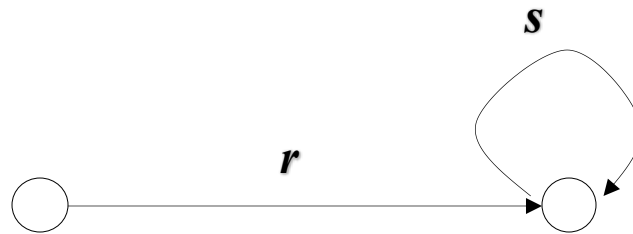
(e) rs 对应的状态转换图



(f) r^* 对应的状态转换图



(g) r^+ 对应的状态转换图



(h) rs^* 对应的状态转换图

图3.8 典型正则表达式对应的状态转换图



3.2.6 正则表达式转换为状态转换图

□转换过程如下：

- 设置一个开始状态和一个终止状态，从开始状态到终止状态引一条标记为待转换表达式的边；
- 检查图中边的标记，如果相应的标记不是字符、 $\emptyset$ 、 ε 或用“|”连接的字符和 ε ，则根据规则(1)-(8)进行替换，直到图中不再存在不满足要求的边。按照习惯，如果一条边上标记的是 $\emptyset$ ，这个边就不用画出来。

称有限自动机 a_1 和 a_2 等价是指

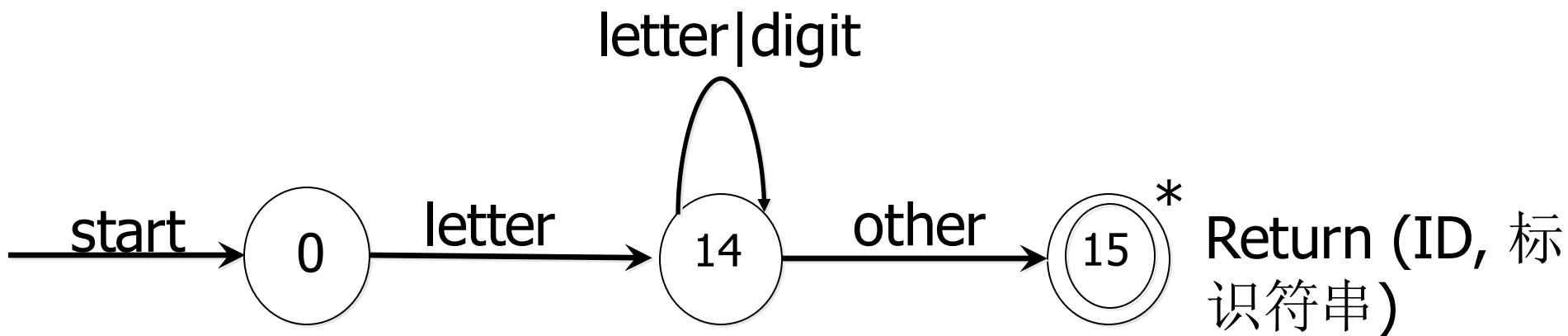
- ☐ A 状态数或有向边数相等
- ☐ B 都是定义在一个字母表上的有限自动机
- ☐ C 状态数和有向边数相等
- ☒ D 所能识别的字符串集合相等



3.3 单词的识别

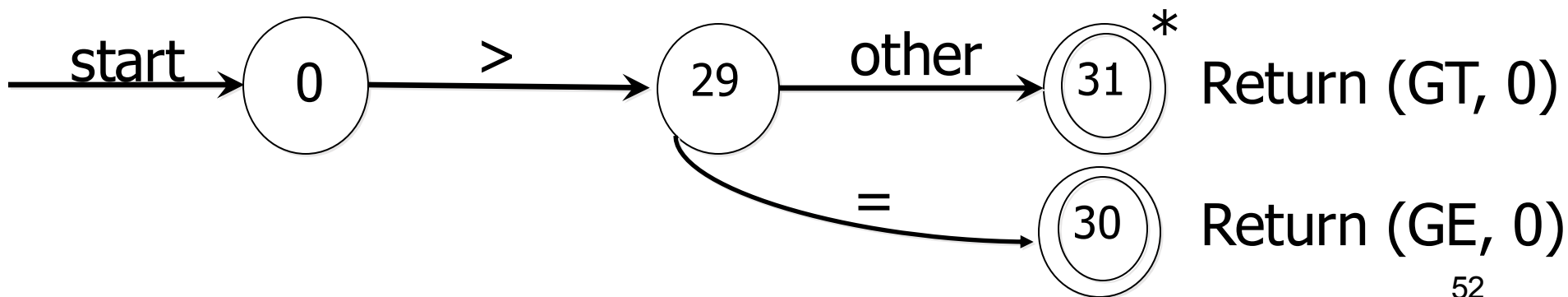
3.3.1 有穷状态自动机与单词识别的关系

- 有穷状态自动机和正则文法等价，考虑到状态转换图的直观性，我们从状态转换图出发来考虑词法分析器的设计。
- 允许在状态转换图的边上标记像digit、letter这样意义明确的符号，other表示例外情况



3.3.1 有穷状态自动机与单词识别的关系

- 考虑到在识别单词的过程中需要执行一些动作，所以将这些动作标记标在基本的状态转换图上。
- 如果到达终止状态，则意味着读入了一个与当前单词无关的字符，由于这个无关字符是下一个单词的开始符号，所以必须回退一个字符。状态上的*表示向前指针必须回退一个字符。





例 3.14 不同进制无符号整数的识别

八进制数: (OCT, 值)

□ oct $\rightarrow 0(0|1|2|3|4|5|6|7)(0|1|2|3|4|5|6|7)^*$

十进制数: (DEC, 值)

□ dec $\rightarrow (1|\dots|9)(0|\dots|9)^* | 0$

十六进制数: (HEX, 值)

□ hex $\rightarrow 0x(0|1|\dots|9|a|\dots|f|A|\dots|F)(0|\dots|9|a|\dots|f|A|\dots|F)^*$



识别不同进制数的状态图

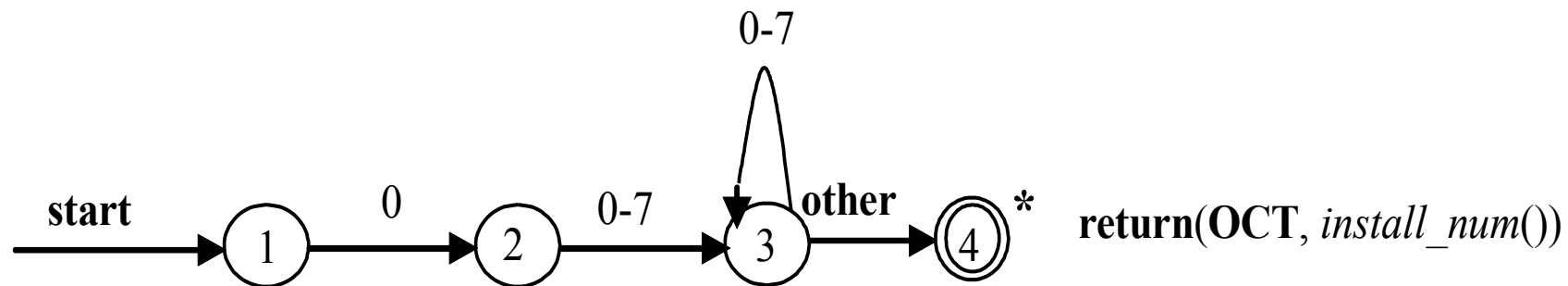


图3.11 识别八进制无符号整数的状态转换图



识别不同进制数的状态图

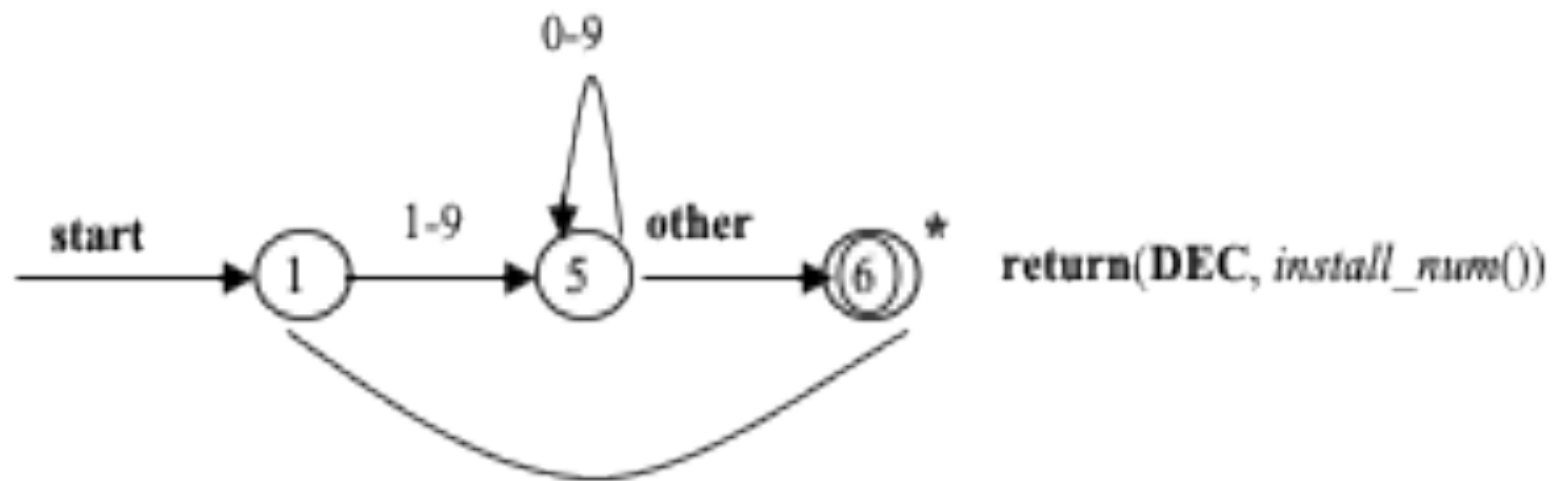


图3.12 识别十进制无符号整数的状态转换图



识别不同进制数的状态图

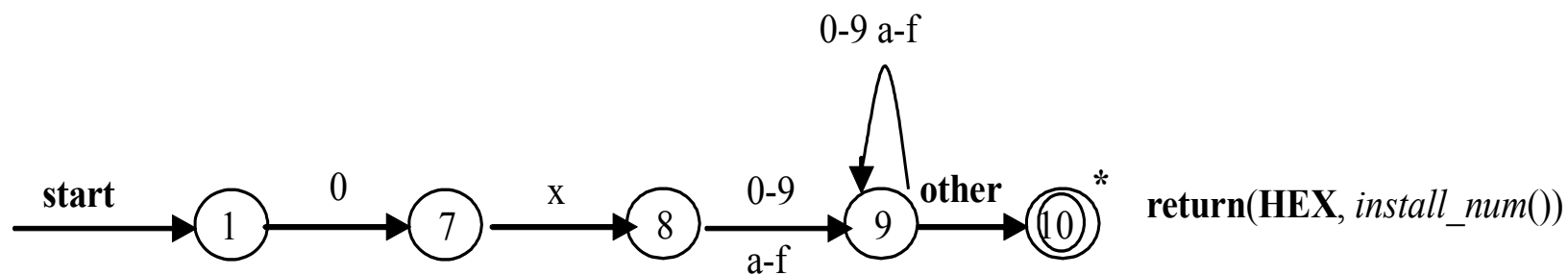


图3.13 识别十六进制无符号整数的状态转换图



识别不同进制数的状态图

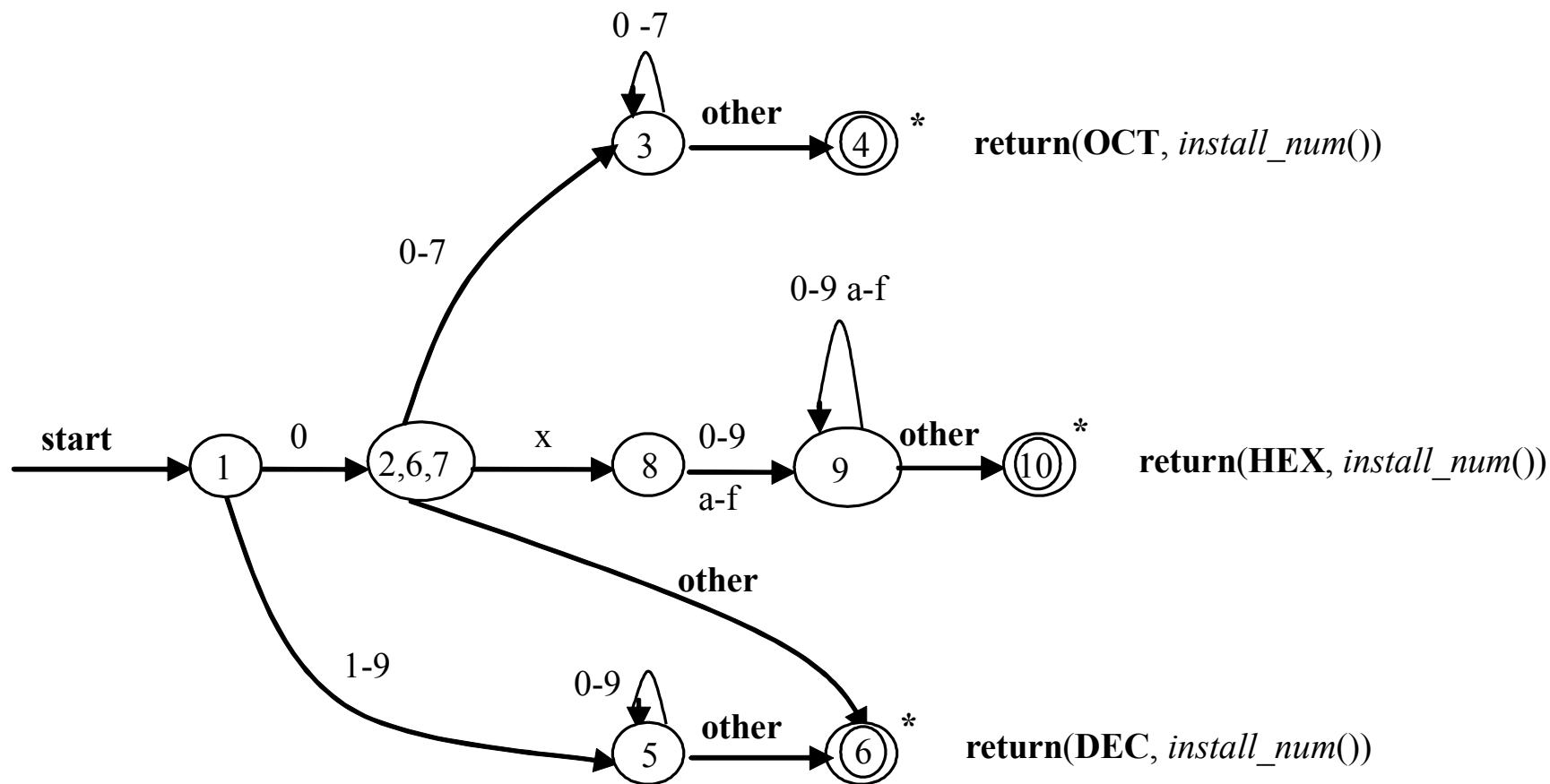


图3.14 识别C语言不同进制无符号整数的状态转换图



3.3.2 单词识别的状态转换图表示

□ $\langle id \rangle \rightarrow \text{letter } \langle id_left \rangle$

□ $\langle id_left \rangle \rightarrow \epsilon | \text{letter } \langle id_left \rangle | \text{digit } \langle id_left \rangle$

□ $\langle int \rangle \rightarrow \text{digit } \langle int_left \rangle$

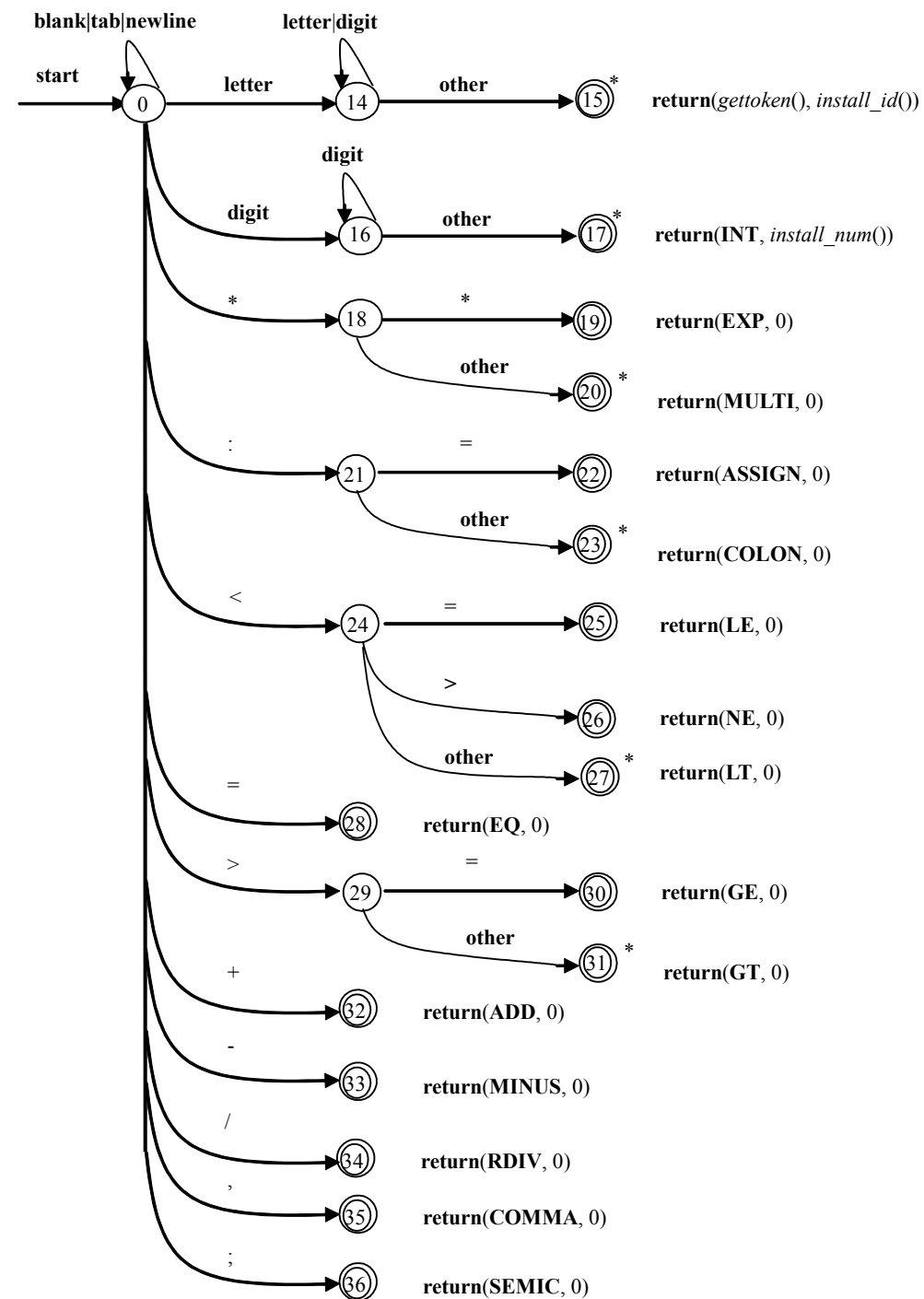
□ $\langle int_left \rangle \rightarrow \epsilon | \text{digit } \langle int_left \rangle$

□ $\langle assignment \rangle \rightarrow :=$

□ $\langle relop \rangle \rightarrow < | <= | = | < > | > | >=$

□ $\langle op \rangle \rightarrow + | - | * | / | **$

□ $\langle delimiter \rangle \rightarrow : | , | ;$





3.3.4 词法分析程序的编写

□ 状态转移图——教材P93图3.15

□ 状态转移图的实现——教材P105图3.22

□ 词法分析程序 *token_scan()*

- 输入：字符流
- 输出：
 - *symbol*: 单词种别
 - *attr*: 属性（全局变量）



数据结构与子例程

□数据结构

- *ch* 字符变量，存放当前读入的输入字符
- *token* 字符串变量，存放构成单词的字符串
- *symbol* 单词种别（词法分析子程序的返回值）
- *attr* 属性（全局变量）

□子例程

- *install_id(token)*:将*token*存入符号表，返回入口指针
- *getchar()*:从输入缓冲区中读入一个字符放入*ch*
- *retract()*:将向前指针回退一个字符，将*ch*置为空白符
- *copytoken()*:返回输入缓冲区中从开始指针*lexeme_beginning*到向前指针*forward*之间的字符串
- *isLetter()* *isalpha()* *isalnum()*

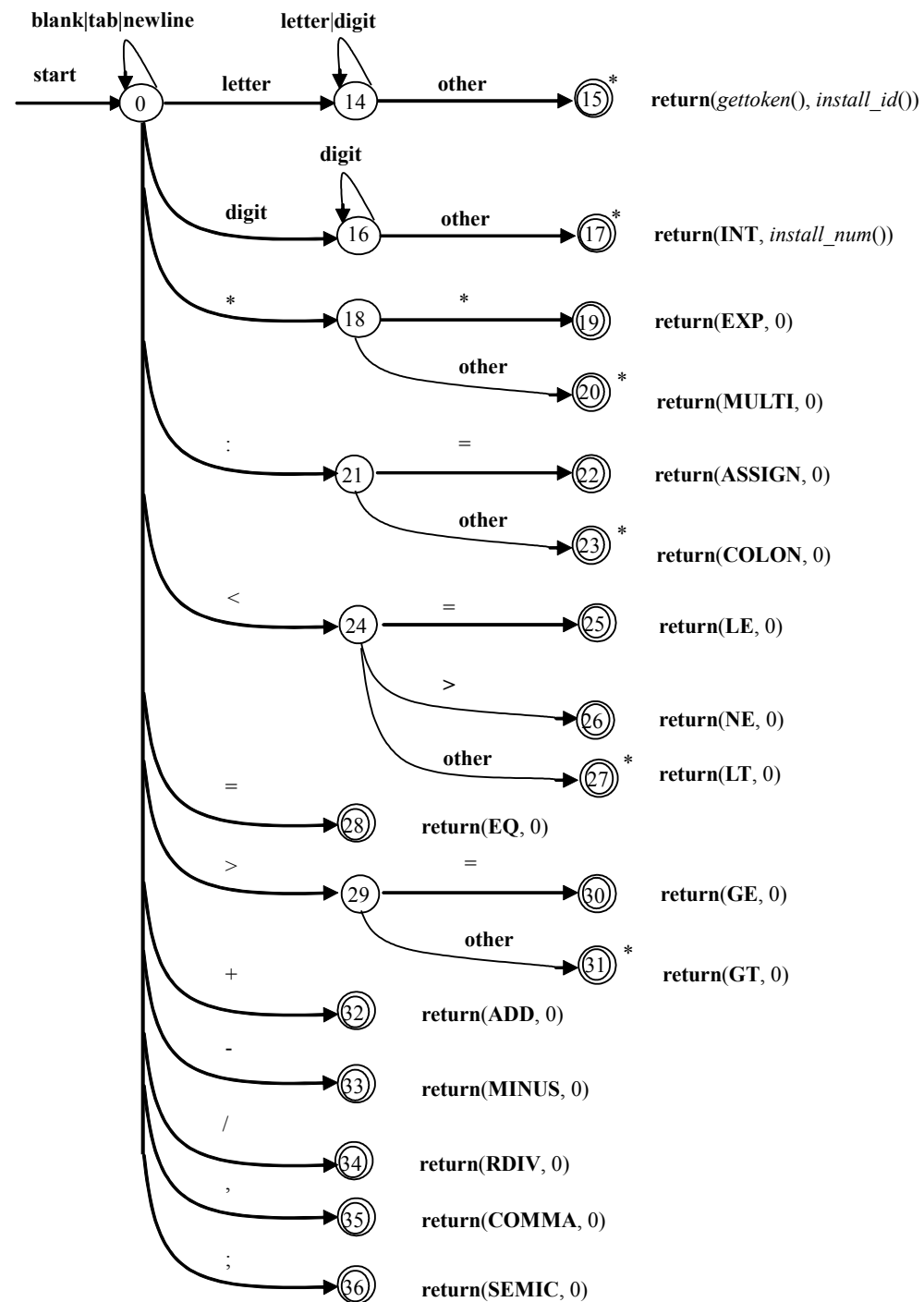




图3.15的状态转换图的实现算法

```
token token_scan()
{  char ch;
   char* token;
   ch = getchar();
   while (ch == blank || ch == tab || ch == newline) {
       ch = getchar();
       lexeme_beginning++;
   }
   if (isalpha(ch)) { ch = getchar();
                     while (isalnum(ch))
                         ch = getchar();
                     retract(1);
                     token = copytoken();
                     return(gettoken(token), install_id(token));
                   }
```



```
else
if (isdigit(ch)) {
    ch = getchar();
    while (isdigit(ch))
        ch = getchar();
    retract(1);
    token = copytoken();
    return(INT, install_num(token));
}
else
switch(ch)
{
    case '*': ch = getchar();
        if(ch == '*') return(EXP, 0);
        else {
            retract(1);
            return(MULTI, 0);
        } break;
```




```
case ':': ch = getchar();
    if(ch == '=') return(ASSIGN, 0);
    else { retract(1); return(COLON, 0);
    } break;
case '<': ch = getchar();
    if(ch == '=') return(LE, 0);
    else if(ch == '>') return(NE, 0);
else { retract(1); return(LT, 0);
    } break;
case '=': return(EQ, 0); break;
case '>': ch = getchar();
    if(ch == '=') return(GE, 0);
    else { retract(1); return(GT, 0);
    } break;
```

```
case '+': return(PLUS, 0); break;
case '-': return(MINUS, 0); break;
case '/': return(RDIV, 0); break;
case ',': return(COMMA, 0); break;
case ';': return(SEMIC, 0); break;
default: error_handle(); break;}return;}
```



需要说明的问题

- 缓冲区预处理，超前搜索
- 关键字的处理，符号表的实现
- Install_id(): 查找效率，算法的优化实现
- 词法错误处理
- 由于高级语言的词组成的集合为3型语言，所以，这里讨论的词法分析技术可以用于处理所有的3型语言，也就是所有的可以用3型文法描述的语言。如：信息检索系统的查询语言、命令语言等

词法分析器不能()。

- ☐ A 过滤源程序中的注释
- ☒ B 发现括号不匹配
- ☐ C 识别出数值常量
- ☐ D 扫描源程序并识别记号

提交



3.4 词法分析程序的自动生成

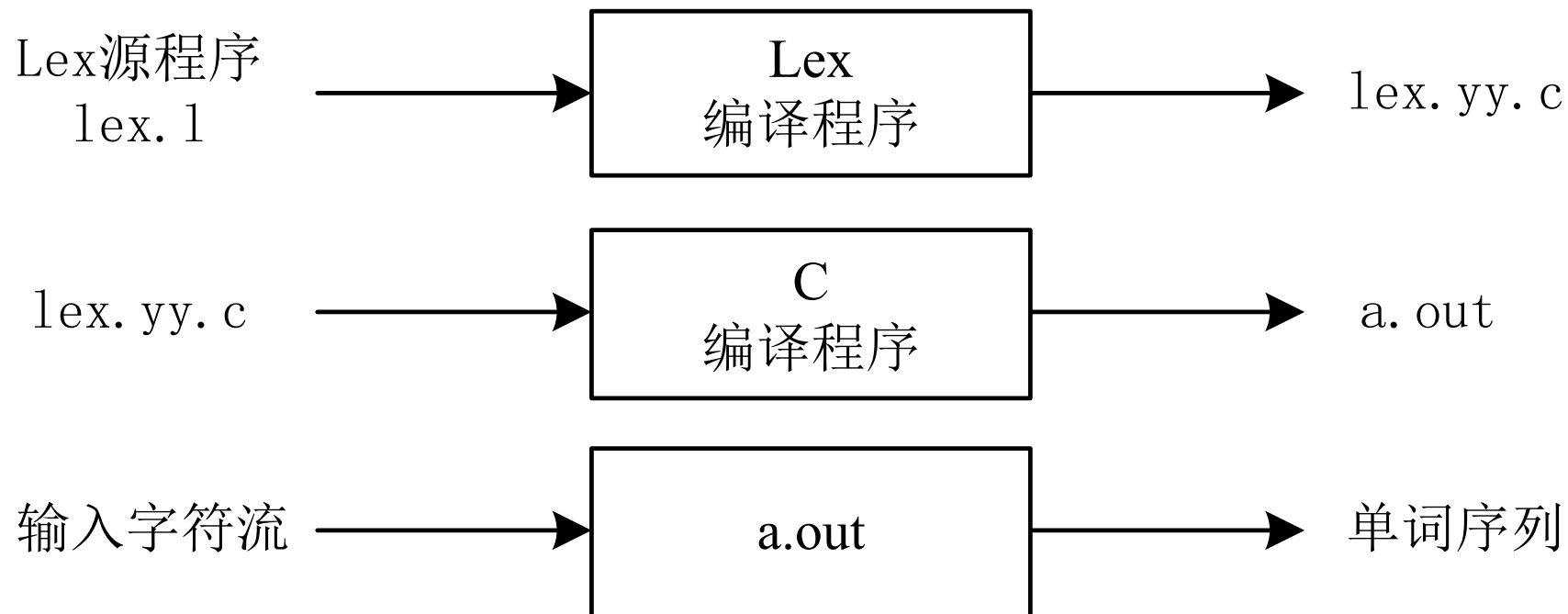


图3.23 利用Lex建立词法分析程序的过程



3.4.1 Lex源程序

声明部分

(正规定义式)

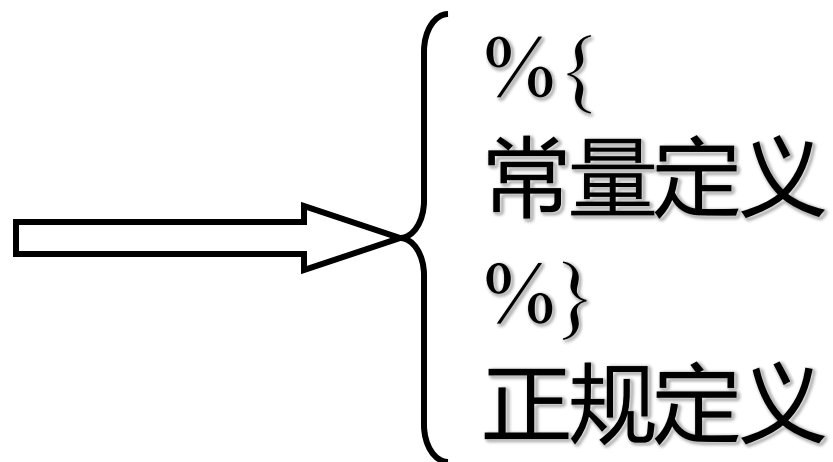
%%

识别规则部分

(识别规则)

%%

辅助过程部分





3.4.1 Lex源程序

1、正规定义式

letter $\rightarrow$ A|B|C|...|Z|a|b|c|...|z

digit $\rightarrow$ 0|1|2|...|9

identifier $\rightarrow$ letter(letter|digit)*

integer $\rightarrow$ digit(digit)*

2、识别规则

正规式	动作描述
-----	------

token1	{action1}
--------	-----------

token2	{action2}
--------	-----------

.....

token _n	{action _n }
--------------------	------------------------

- **%{**
- **#include <stdio.h>**
- **#include "y.tab.h"**
- **#define ID** 1
- **#define INT** 2
- **#define EXP** 3
- **#define MULTI** 4
- **#define COLON** 5
- **#define EQ** 6
- **#define NE** 7
- **#define LE** 8
- **#define GE** 9
- **#define LT** 10
- **#define GT** 11
- **#define PLUS** 12
- **#define MINUS** 13
- **#define RDIV** 14
- **#define COMMA** 15
- **#define SEMIC** 16
- **#define RELOP** 17
- **#define ASSGIN** 18
- **int line_no = 1;** **%}**

- **delim** **[\t\n]**
- **ws** **[delim]+**
- **letter** **[a-zA-Z]**
- **digit** **[0-9]**
- **id** **{letter}({letter}|{digit})***
- **number** **{digit}+**
- **%%**
- **{ws}** **;**
- **begin** **return(BEGIN);**
- **end** **return(END);**
- **if** **return(IF);**
- **then** **return(THEN);**
- **else** **return(ELSE);**
- **do** **return(DO);**
- **program** **return(PROGRAM);**
- **{id}** **{yyval = install_id(); return(ID);}**
- **{number}** **{yyval = install_num(); return(INT);}**

红色：声明部分的基本定义部分

绿色：声明部分的进阶定义部分

蓝色：识别规则部分

- "<" {yyval =LT; return(RELOP);}
- "<=" {yyval =LE; return(RELOP);}
- "=" {yyval =EQ; return(RELOP);}
- ">" {yyval =GT; return(RELOP);}
- ">=" {yyval =GE; return(RELOP);}
- "<>" {yyval =NE; return(RELOP);}
- "+" return(PLUS);
- "-" return(MINUS);
- "*" return(MULTI);
- "/" return(RDIV);
- "***" return(EXP);

- ":" return(COLON);
- "==" return(ASSGIN);
- "," return(COMMA);
- ";" return(SEMIC);
- \n line_no++;
- . { fprintf(stderr, "'%c' (0%o): illegal charcter at line %d\n", yytext[0], yytext[0], line_no); }
- %%
- install_id()
- {.....}
- install_num()
- {.....}

蓝色：识别规则部分
黑色：辅助过程部分


```

% {
    /* 声明部分的开始,这一部分涉及用到的文件表、符号
       常量声明、变量声明 */

#include <stdio.h>
#include "y.tab.h"

#define ID      1      /* 定义标识符等的种别码 */
#define INT     2
#define EXP     3
#define MULTI   4
#define COLON   5
#define EQ      6
#define NE      7
#define LE      8
#define GE      9
#define LT     10
#define GT     11
#define PLUS    12
#define MINUS   13
#define RDIV    14
#define COMMA   15
#define SEMIC   16
#define RELOP   17
#define ASSIGN  18

int line_no = 1;

% }

/* 基本定义部分结束 */

delim      [ \t\n ]      /* 正则定义式的开始,定义 delim 代表制表符、换行符 */
ws         [ delim ] +    /* 定义 ws 代表至少一个制表符、换行符 */
letter     [ a-zA-Z ]    /* 定义 letter 代表小写和大写字母 */
digit      [ 0-9 ]       /* 定义 digit 代表数字 */

id         { letter } ( { letter } | { digit } ) * /* 定义 id 为标识符,是以字母开头的字母数字串 */
number     { digit } +    /* 定义 number 代表数,是非空数字串 */

% %
/* 声明部分结束,识别规则部分开始 */

{ ws } ; /* 识别出的 ws 表示空白,要忽略掉 */

begin      return( BEGIN ); /* 识别单词“begin” */
end        return( END );
if         return( IF );

```

```

then       return( THEN );
else       return( ELSE );
do         return( DO );
program    return( PROGRAM );
{ id }     { yyval = install_id(); return( ID ); } /* 识别出标识符 id,此时需要执行相应的语义动作 */

{ number } { yyval = install_num(); return( INT ); }
" < "     { yyval = LT; return( RELOP ); }
" < = "    { yyval = LE; return( RELOP ); }
" = "     { yyval = EQ; return( RELOP ); }
" > "     { yyval = GT; return( RELOP ); }
" > = "    { yyval = GE; return( RELOP ); }
" < > "    { yyval = NE; return( RELOP ); }
" + "     return( PLUS );
" - "     return( MINUS );
" * "     return( MULTI );
" / "     return( RDIV );
" * * "    return( EXP );
" : "     return( COLON );
" : = "    return( ASSIGN );
" , "     return( COMMA );
" ; "     return( SEMIC );
\n        line_no + + ;

. { fprintf( stderr, "' %c ' (0%o): illegal character at line %d\n", yytext[0], yytext[0], line_no ); }
% %
/* 识别规则部分结束,辅助过程部分开始 */

install_id()
{
...
/* 该过程负责将单词填入符号表。yytext 指向单词的第一个字符,yylen 表示单词的长度。
   如果该单词已经出现在符号表中,则返回指向该单词所在表项的指针,否则为它创建一个新表项,将该单词填入该表项中,并返回指向新表项的指针 */
}

install_num()
{
...

```

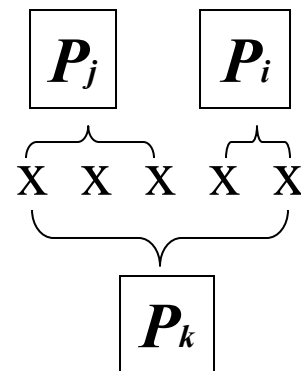


LEX二义性问题的两条原则

如:begin:=

1.最长匹配原则

在识别单词过程中，有一字符串
根据最长匹配原则，应识别为这是
一个符合 P_k 规则的单词，
而不是 P_j 和 P_i 规则的单词。



2.优先匹配原则

如果有一字符串有两条规则可以同时匹配时，那么用规则
序列中位于前面的规则相匹配，所以排列在最前面的规则优先
权最高。



3.4.2 Lex的实现原理

Lex的功能是根据Lex源程序构造一个词法分析程序，
该词法分析器实质上是一个有穷自动机。

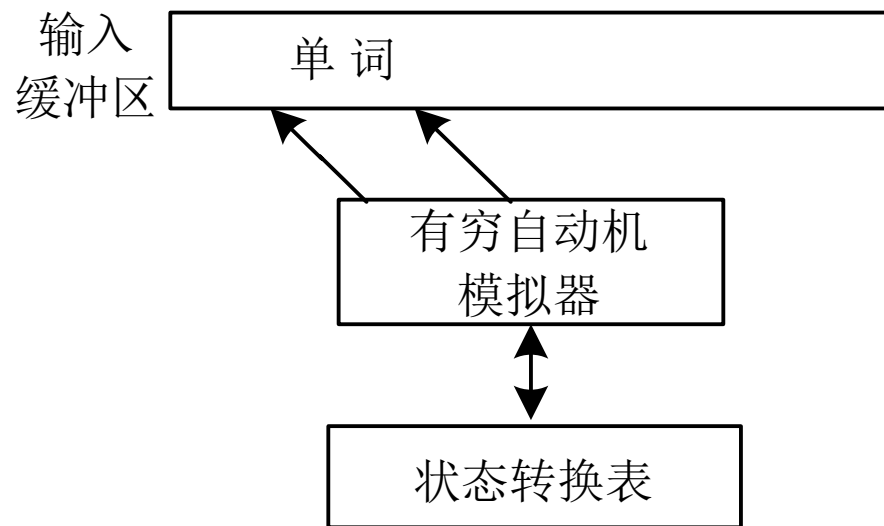


图 3.24 Lex生成的词法分析器结构

Lex的功能是根据Lex源程序生成状态转换矩阵和控制程序



三点说明

- 以上是Lex的构造原理，虽然是原理性的，但据此就不难将Lex构造出来。
- 所构造出来的Lex是一个通用的工具，用它可以生成各种语言的词法分析程序，只需要根据不同的语言书写不同的LEX源文件就可以了。
- Lex不但能自动生成词法分析器，而且也可以产生多种模式识别器及文本编辑程序等

本章小结

- 词法分析器接收表示源程序的“平滑字符流”，输出与之等价的单词序列；
- 单词被分成多个种类，并被表示成(种别，属性值)的二元组形式；
- 为了提高效率，词法分析器使用缓冲技术，而且在将字符流读入缓冲区分时，是经过剔除注解、无用空白符等预处理后的结果；

本章小结

- 单词的识别相当于正则语言的识别;
- 词法的等价描述形式有正则文法、有穷状态自动机、正则表达式, 其中有穷状态自动机可以用状态转换图表示;
- 实现词法分析器时状态转换图是一个很好的设计工具, 根据该图, 容易构造出相应的分析程序;
- 使用恰当的形式化描述, 可以实现词法分析器的自动生成, Lex就是一种自动生成工具。